Project Title: IUT Laundry Management System
Course Code: SWE 4404
Course Title: Software Project Lab II (SPL-II)
Institution: Islamic University of Technology (IUT)
Supervisor(s): Farzana Tabassum

Team Members:
- Ibrahim Inyass Youssouf (230042148)
- Tahmid Hafiz (230042136)

Source Repository: [tahmidhafiz516-bot/SPL-II_Repository: IUT Laundry Management System]

Table of Contents:
1. Executive Summary
2. Problem Domain and Background
3. Literature and Existing Solutions
4. Requirements (SRS Session)
5. System Design (Design Session)
6. Implementation (Implementation Session)
7. Tools, Version Control, Testing, and CI/CD
8. Professional, Ethical, and Legal Considerations
9. Project Management and Feasibility
10. Teamwork and Individual Contributions
11. Results, Limitations, and Future Work
12. Glossary and References

Executive Summary

The IUT Laundry Management System is a software project designed to improve the management of laundry services at the Islamic University of Technology. The existing laundry process involves several manual activities, including submitting clothes, recording customer information, tracking laundry progress, and managing payments. As the number of students using the laundry service increases, these activities become increasingly difficult to manage efficiently.

The absence of a centralized digital system creates several problems. Students and faculty members have limited information about the progress of their laundry requests, communication between customers and laundry staff can be difficult, and manually maintaining payment and order records increases the possibility of errors. There is also a risk of clothes being delayed, misplaced, or incorrectly returned.

The proposed system addresses these problems by introducing a centralized application through which customers and laundry personnel can manage laundry-related activities. It supports four primary user roles: students, faculty members, laundry staff, and the laundry manager.

Students and faculty members can register for accounts, submit laundry requests, select their preferred services, and monitor order progress. Laundry staff can receive requests and update their processing status. The laundry manager is responsible for managing customer accounts and overseeing administrative activities. Additional functionality addresses complaints, damaged or missing garments, rewashing requests, and delivery verification.

The implementation uses Java as its primary programming language, MySQL for persistent data storage, JDBC for database communication, and Maven for dependency management. The application is organized into three principal packages: Users and Access, Laundry Operations, and Customer Support and Communication. This structure separates responsibilities and makes the codebase easier to maintain.

The current implementation provides a foundation for managing users, laundry orders, garments, complaints, notifications, and delivery-related activities. Passwords are processed using salted PBKDF2-based hashing rather than being stored as ordinary plaintext.

The project has progressed from requirements analysis and system design to a Java-based implementation. However, the implementation should not be considered a fully production-ready laundry management platform. Live MySQL integration, complete end-to-end testing, payment processing, and external notification services have not been fully verified or implemented.

Overall, the project demonstrates how software engineering principles, object-oriented programming, database management, and modular design can be applied to a practical university service-management problem.

Problem Domain and Background

1. Application Domain

   The project belongs to the domain of service management and information systems, specifically laundry-service management.

   The IUT laundry service handles clothing submissions, processing, customer records, and the return of completed laundry. Managing these activities manually becomes increasingly challenging as the number of customers and transactions grows.

   The system aims to provide a centralized method of recording and managing these activities.

   The primary users are:

   - *Students*: Submit laundry requests, select services, track progress, and report problems.
   - *Faculty Members*: Use the laundry service and monitor their submitted orders.
   - *Laundry Staff*: Receive orders, process garments, and update order statuses.
   - *Laundry Manager*: Manage customer accounts and oversee laundry operations.

2. Problem Statement

   The main problems identified during requirements analysis include:

   - Laundry requests are handled through a manual process.
   - Students and faculty members cannot conveniently track their laundry status.
   - Communication between customers and laundry personnel is inefficient.
   - Payment records are difficult to manage manually.
   - Clothes may be delayed, misplaced, damaged, or incorrectly returned.
   - Maintaining customer and order histories requires additional administrative effort.

   The project addresses these problems by introducing a centralized digital management system.

3. Fundamental Computing Concepts

   Several fundamental computing concepts are directly relevant to the project.

   Object-Oriented Programming

   The system uses object-oriented programming to represent real-world entities.

   Examples include:

   - *User*: Represents a general system user.
   - *Student* and *Faculty*: Represent customer categories.
   - *LaundryOrder*: Represents an individual laundry request.
   - *Garment*: Represents clothing associated with an order.
   - *Complaint*: Represents a customer-reported issue.
   - *Delivery*: Represents the return of completed laundry.

Inheritance allows related user types to share common properties and behavior.

Encapsulation helps keep data and the operations associated with it within individual classes.

Data Structures

The system works with collections of users, orders, garments, complaints, and notifications.

Java collections are suitable for handling these records in memory during application operations.

The database also uses relational structures to organize persistent records and establish relationships between users, orders, and other entities.

Database Management

MySQL provides persistent storage for the system.

The database is designed around related entities rather than storing all information in a single table. This supports more organized data management and reduces unnecessary duplication.

Primary keys identify records, while foreign keys establish relationships between related records.

Design Principles

The implementation follows a modular organization based on separation of concerns.

The three principal packages separate account management, laundry operations, and customer support.

This makes it possible to modify one functional area without unnecessarily affecting unrelated areas.

4. Specialist Knowledge Required

   The application requires knowledge from several specialist areas.

   Database engineering: Understanding relational databases, SQL queries, primary keys, foreign keys, and JDBC is necessary for maintaining consistent customer and laundry records.

   Information security: User authentication and password protection are important because the application stores account credentials and personal information.

   Service workflow management: Laundry processing involves several stages, so the system must maintain order statuses and coordinate activities between customers and laundry personnel.

Software engineering: Requirements analysis, UML modeling, modular architecture, testing, and maintenance are necessary to develop a reliable application.

These areas are important because the application must do more than store information. It must also represent the actual workflow of a laundry service.

Literature and Existing Solutions

1. Existing Approaches

   Laundry management can be performed through manual recordkeeping, spreadsheets, or dedicated digital management applications.

   Manual Management

   In a manual system, customer details and laundry submissions are recorded by personnel. Customers generally need to communicate directly with staff to obtain updates.

   Although this approach requires little initial technical infrastructure, it becomes increasingly difficult to maintain as transaction volume grows.

   Its limitations include:

   - Repetitive administrative work.
   - Difficulty retrieving previous records.
   - Limited visibility of order progress.
   - Greater dependence on manual communication.
   - Increased possibility of recording errors.

   Spreadsheet-Based Management

   Spreadsheets provide a basic digital alternative to paper records.

   They can organize customer information and transaction histories, but they do not automatically provide a complete customer-facing ordering and tracking system.

   They also offer limited support for managing multiple user roles and coordinating different processing stages.

   Dedicated Laundry Management Applications

   Dedicated laundry-management applications generally provide features such as order management, customer records, service selection, transaction tracking, and administrative reporting.

   These applications demonstrate the usefulness of centralized digital management.

However, a university laundry service has specific requirements involving student and faculty accounts, account verification, garment handling, and communication with laundry personnel.

The IUT Laundry Management System focuses on these institutional requirements.

2. Technical Sources

Several technical sources informed the implementation approach.

MySQL Connector/J Documentation

MySQL Connector/J provides the JDBC driver required for Java applications to communicate with MySQL databases. It supports the database integration approach selected for this project.

Oracle Java Documentation

Java's JDBC interfaces provide the mechanisms required to establish database connections, execute SQL statements, and retrieve query results.

OWASP Password Storage Guidance

OWASP recommends using dedicated password-hashing algorithms rather than storing passwords in plaintext. The project uses salted PBKDF2-based password processing.

3. Comparison with the Proposed System

Area	**Manual/Basic Systems**	**IUT Laundry Management System**
Customer Records	Manual or spreadsheet-based	Centralized database
Accounts Management	Primary manual	User accounts with manager verification
Laundry Requests	Manual submission	Application-based requests
Order Tracking	Requires communication with staff	Recorded order statuses
Garment Records	Manual records	Garment-related data models
Complaints	Informal reporting	Complaint management functionality
Rewashing	Requires manual coordination	Rewash request functionality
Notifications	Direct Communication	Database-stored notifications

4. Project Contribution

The project adapts general laundry-management concepts to the requirements of IUT.

Its main contribution is the combination of customer account management, laundry processing, order tracking, garment management, complaint handling, and delivery-related activities in one application.

Rather than attempting to develop a commercial laundry platform with extensive external integrations, the project focuses on the core operational requirements of a university laundry service.

Requirements (SRS Session)

1. Stakeholders

   The primary stakeholders identified in the SRS are:

Stakeholder	Responsibilities and Needs
Students	Register, submit laundry requests, monitor orders, and report problems
Faculty Members	Submit requests and monitor laundry progress
Laundry Staff	Receive requests and update processing statuses
Laundry Manager	Verify accounts, manage administrative activities, and oversee operations
System Administrator	Maintain the application and its technical environment

2. Functional Requirements

   The functional requirements describe what the system should do.

ID	Functional Requirement	Description
FR-01	User Registration	Allow students and faculty members to submit registration information
FR-02	Account Management	Allow the manager to review and approve customer accounts
FR-03	Authentication	Allow registered users to access the system securely
FR-04	Laundry Request	Allow customers to submit laundry requests
FR-05	Service Selection	Allow customers to select washing and ironing or ironing-only services
FR-06	Status Tracking	Allow customers to monitor laundry progress
FR-07	Laundry History	Maintain records of previous laundry requests
FR-08	Laundry Processing	Allow staff to manage submitted laundry orders
FR-09	Issue Reporting	Allow customers to report damaged or missing garments
FR-10	Rewash Requests	Allow customers to request another wash when necessary
FR-11	Delivery Management	Maintain delivery and collection-related information
FR-12	Notification Management	Store notifications related to system activities
FR-13	Administrative Management	Support manager-level oversight of laundry operations

3. Non-Functional Requirements

   Non-functional requirements describe the expected quality of the system.

ID	Requirement	Description
NFR-01	Security	Protect user credentials and restrict access to administrative functions
NFR-02	Reliability	Maintain accurate and consistent records
NFR-03	Usability	Provide a straightforward interface for different user categories
NFR-04	Maintainability	Organize the application into manageable modules
NFR-05	Performance	Support efficient retrieval and updating of records
NFR-06	Data Integrity	Maintain valid relationships between users, orders, garments, and complaints
NFR-07	Availability	Make the application accessible when the required application and database services are running

4. Requirements Traceability

Requirement	Stakeholder Need	Related User Story
FR-04: Submit Laundry Request	Students and faculty need to submit clothes for processing	US-08: Faculty member selects a preferred laundry service
FR-06: Track Status	Customers need to know when their clothes will be ready	US-11: Student checks laundry status
FR-07: View History	Students need access to previous laundry requests	US-27: Student views laundry history
FR-02: Manage Accounts	Manager needs to verify customer registrations	US-03: Manager reviews registration requests
FR-09: Report Issues	Customers need to report damaged or missing garments	US-20: Student reports an issue

5. Contextual and Sustainable Development Considerations

   The system is intended for use in a university environment.

   Several contextual factors influence its requirements:

   - Students and faculty members have different institutional identities.
   - Account approval is required to prevent unauthorized registrations.
   - Laundry staff need a practical method of updating processing information.
   - Customers need clear information about their submitted garments.
   - The system should be maintainable within the technical resources available to the university.

   From a sustainability perspective, digital records can reduce dependence on paper-based administration. Better tracking may also reduce unnecessary communication and help identify misplaced garments.

System Design (Design Session)

1. Architectural Design

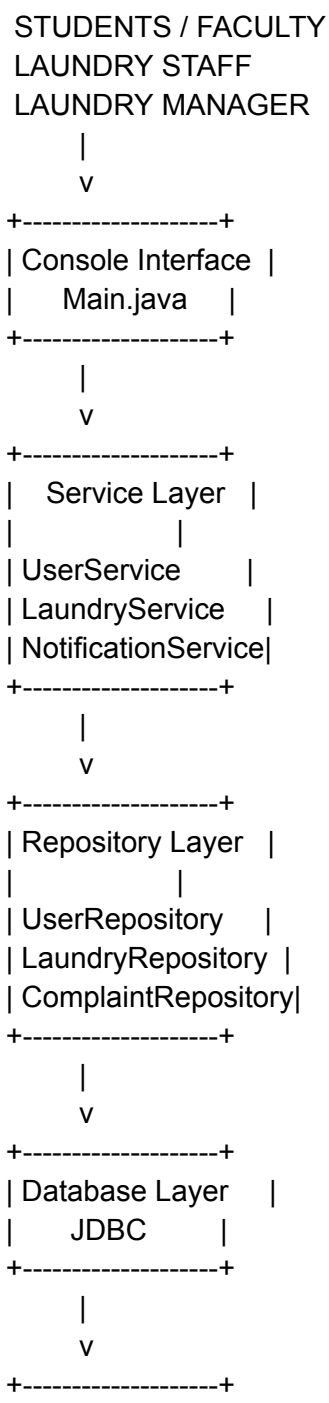
   The system follows a modular, layered architecture.

The main layers are:

1. User Interface Layer
2. Service Layer
3. Repository/Data Access Layer
4. Database Layer

The application currently uses a console-based interface.

Architecture Diagram



```
|  MySQL Database  |
|   iut_laundry    |
+------------------+
```

The interface communicates with services, services handle application logic, repositories perform database operations, and JDBC provides the connection to MySQL.

2. Component Design

The system is divided into three major functional components.

Users and Access

Responsible for account management and authentication.

Important classes:

- *User*
- *Student*
- *Faculty*
- *LaundryManager*
- *LaundryStaff*
- *UserRole*
- *UserRepository*
- *UserService*
- *PasswordUtil*
- *Database*

Laundry Operations

Responsible for managing laundry requests and processing activities.

Important classes:

- *Garment*
- *LaundryOrder*
- *Delivery*
- *RewashRequest*
- *LaundryRepository*
- *LaundryService*

Customer Support and Communication

Responsible for complaints and notifications.

Important classes:

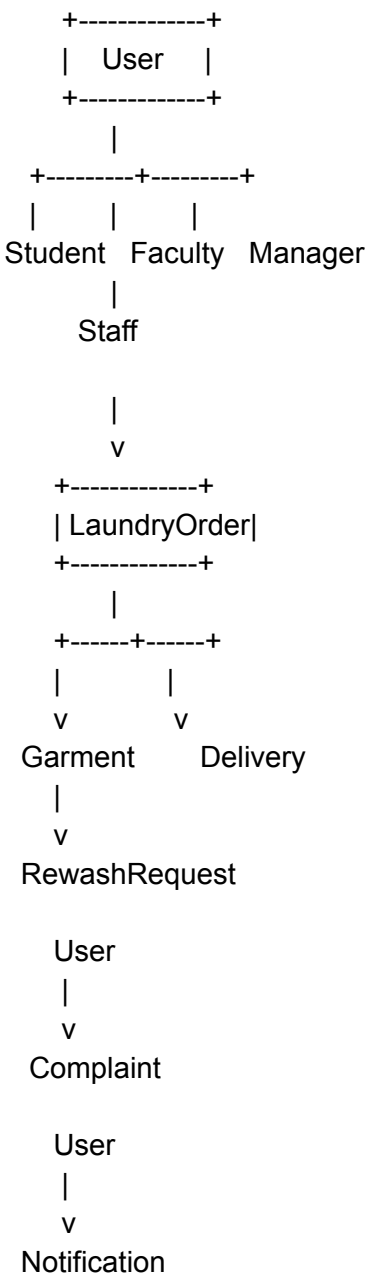
- *Complaint*

- *Notification*
- *ComplaintRepository*
- *NotificationService*
- *Main*

3. Class and Data Model

The object-oriented model represents the major entities involved in laundry management.

A simplified relationship diagram is shown below.



The diagram illustrates the main conceptual relationships rather than every database association or multiplicity.

4. Database Design

   The MySQL database is named:

   `iut_laundry`

   The implementation's schema includes the following tables:

Table	Purpose
users	Stores account information and user roles
laundry_orders	Stores laundry order information
garments	Stores garment-related records
complaints	Stores customer complaints
rewash_requests	Stores requests for additional washing
deliveries	Stores delivery-related information
notifications	Stores system notifications

   Primary and foreign keys are used to establish relationships between relevant records.

   The schema is provided in:

   `database/schema.sql`

   The database must be created before running the application.

5. Design Alternatives

   Two plausible architectural approaches were considered for evaluating the system design.

   ### Alternative 1: Monolithic Application

   A monolithic implementation could place the interface, business logic, and database operations in a small number of classes.

   Advantages:

   - Easier to begin developing.
   - Requires less initial configuration.
   - Suitable for a small prototype.

   Disadvantages:

   - Responsibilities become mixed.
   - Changes can affect unrelated functionality.
   - Maintenance becomes difficult as features increase.

   ### Alternative 2: Modular Layered Application

The selected implementation separates functionality into packages and uses service and repository classes.

Advantages:

- Better separation of responsibilities.
- Easier maintenance and debugging.
- Supports independent development of functional areas.
- Makes database operations easier to organize.
- Suitable for the current project size.

Disadvantages:

- Requires more classes and initial organization.
- Introduces additional dependencies between layers.
- Requires careful package and interface management.

Comparison

Evaluation Criteria	Monolithic Design	Modular Layered Design
Initial development	Simpler	More structured
Maintainability	Lower as project grows	Better separation
Scalability	Limited flexibility	Easier to extend
Resource requirements	Low	Low to moderate
Development complexity	Lower initially	Higher initially
Suitability for project	Acceptable prototype	Better long-term structure

The modular layered design was selected because the project contains several related functional areas that benefit from separation.

A microservices architecture was not selected because the project does not require the infrastructure and deployment complexity associated with independently distributed services.

Implementation (Implementation Session)

This section describes the implemented software, its organization, communication between components, and the reasoning behind its class structure.

1. Technologies Used

   The implementation uses:

Technology	Purpose
Java	Main programming language
Java 17	Configured project language/runtime target
IntelliJ IDEA	Development environment
Maven	Dependency and build management

MySQL	Persistent database
JDBC	Java-to-MySQL communication
MySQL Connector/J	MySQL JDBC driver

The project is organized as a Maven-based Java application.

2. Project Structure

The codebase is divided into three principal packages.

```
IUT-Laundry-Management-System/
│
├── pom.xml
│
├── README.md
│
└── code/
        ├── database/
        │   └── schema.sql
        │
        ├── usersaccess/
        │   ├── Database.java
        │   ├── UserRole.java
        │   ├── PasswordUtil.java
        │   ├── User.java
        │   ├── Student.java
        │   ├── Faculty.java
        │   ├── LaundryManager.java
        │   ├── LaundryStaff.java
        │   ├── UserRepository.java
        │   └── UserService.java
        │
        ├── laundryoperations/
        │   ├── Garment.java
        │   ├── LaundryOrder.java
        │   ├── Delivery.java
        │   ├── RewashRequest.java
        │   ├── LaundryRepository.java
        │   └── LaundryService.java
        │
        └── customersupportcommunication/
            ├── Complaint.java
            ├── Notification.java
            ├── ComplaintRepository.java
            ├── NotificationService.java
            └── Main.java
```

Reason for the Structure

The three packages represent distinct functional responsibilities.

Users and Access handles user identity, authentication, and account-related database operations.

Laundry Operations handles orders, garments, deliveries, and rewashing.

Customer Support and Communication handles complaints and notifications, while `Main.java` provides the console application's entry point.

This organization follows the principle of separation of concerns.

It also makes it easier to locate classes and identify which part of the application is responsible for a particular operation.

3. Communication Between Components

The application uses direct Java method calls between its layers.

It does not use HTTP requests, REST APIs, JSON messages, or an external messaging service.

Frontend to Backend Communication

The console interface receives user input and invokes the appropriate service methods.

For example, when a customer submits a laundry order:

1. The customer selects the relevant menu option.
2. The interface collects order information.
3. The service layer receives the input.
4. The service layer performs application-level operations.
5. The repository layer stores the information in MySQL.
6. The result is returned to the interface.

The data is exchanged through Java objects and method parameters rather than network-based DTO messages.

Backend to Database Communication

The repository classes use JDBC to communicate with MySQL.

The general communication process is:

Service Method
|
v

Repository Method
|
v
JDBC Connection
|
v
SQL Query / PreparedStatement
|
v
MySQL Database
|
v
ResultSet / Update Result
|
v
Repository
|
v
Service
|
v
Console Interface

The database connection is managed by the `Database` class.

The project uses the following environment variables:

- *IUT_DB_URL*
- *IUT_DB_USER*
- *IUT_DB_PASSWORD*

The default database URL is:

`jdbc:mysql://localhost:3306/iut_laundry?useSSL=false&serverTimezone=UTC`

The MySQL Connector/J dependency provides the JDBC driver.

4. Data Classes Exchanged Between Components

   The application uses model classes to represent information exchanged between services and repositories.

Data Class	Information Represented	Related Component
User	User identity and account information	Users and Access
Student	Student-specific information	Users and Access
Faculty	Faculty-specific information	Users and Access
LaundryOrder	Order information and processing status	Laundry Operations

Garment	Garment-related information	Laundry Operations
Delivery	Delivery information	Laundry Operations
RewashRequest	Rewashing information	Laundry Operations
Complaint	Customer-reported problems	Customer Support
Notification	Notification information	Customer Support

5. Example End-to-End Feature: Submitting a Laundry Order

   The following example illustrates how the components communicate when a customer submits a laundry request.

   Step 1: User Input

   The customer accesses the console interface and selects the option to create a laundry order.

   Step 2: Service Processing

   The interface passes the relevant customer and order information to *LaundryService*.

   Step 3: Repository Operation

   LaundryService invokes the relevant operation in *LaundryRepository*.

   Step 4: Database Insertion

   The repository uses JDBC to execute the necessary SQL statements.

   The order information is stored in the `laundry_orders` table, while associated garment information is stored in the `garments` table.

   Step 5: Response

   The service returns the result to the interface, which displays the corresponding success or failure message.

   This flow demonstrates the separation between user interaction, business logic, and database operations.

6. Division of Classes and Methods

   The classes are divided according to their responsibilities.

   - *UserRepository* handles user-related persistence.
   - *UserService* handles user-related application logic.
   - *LaundryRepository* handles laundry-related database operations.
   - *LaundryService* manages laundry-related application operations.
   - *ComplaintRepository* manages complaint persistence.
   - *NotificationService* manages notification-related operations.

This division follows the Single Responsibility Principle as a design objective.

It also reduces the need to place unrelated operations inside a single large class.

Method Organization

Methods are intended to perform individual operations, such as:

- Registering users.
- Authenticating users.
- Creating laundry orders.
- Updating order statuses.
- Recording complaints.
- Creating rewash requests.
- Managing delivery verification.

This improves readability and makes individual operations easier to maintain and test.

7. Lines of Code Analysis

Measurement	Result
Total Java source lines	969
Total number of Java classes	321
Maximum lines in a method	
Average lines per method	
Maximum lines in a class	174
Average lines per class	46.14

8. Implemented Features

The current implementation includes the following functionality:

Feature	Implementation Status
User registration	Implemented
User authentication	Implemented
Password hashing	Implemented
Manager account approval	Implemented
User role management	Implemented
Laundry order management	Implemented
Garment records	Implemented
Laundry status management	Implemented
Complaint management	Implemented
Rewash request management	Implemented
Delivery records	Implemented
Delivery verification	Implemented
Database-stored notifications	Implemented

9. Refactoring After the Implementation Session

   The development process involved organizing the Java code into separate packages and correcting package and import declarations.

   The package structure was standardized to:

   - *usersaccess*
   - *laundryoperations*
   - *customersupportcommunication*

   This was necessary because Java reserves package names beginning with *java*. for its own platform libraries.

   The database connection and Maven configuration were also reviewed during troubleshooting.

   The MySQL Connector/J dependency was included through Maven rather than relying solely on a manually added library.

   These changes were intended to improve project organization, dependency management, and compatibility with IntelliJ IDEA.

Tools, Version Control, Testing, and CI/CD

1. Development Tools

   Java

   Java was selected because its object-oriented features support the modeling of users, orders, garments, complaints, and other system entities.

   An alternative would have been Python, which could provide faster initial development. However, Java was selected for its suitability for object-oriented coursework and compatibility with the planned implementation.

   IntelliJ IDEA

   IntelliJ IDEA provides Java development, project organization, debugging, and Maven integration.

   An alternative would have been Eclipse. IntelliJ was used as the project's primary development environment.

   MySQL

   MySQL provides relational database storage for customer and laundry information.

   An alternative would have been PostgreSQL. Both support relational data management, but MySQL was selected for this implementation.

Maven

Maven manages dependencies and the project build configuration.

It also allows the MySQL Connector/J dependency to be declared in `pom.xml`.

Without a dependency manager, libraries would need to be managed manually, making project setup and collaboration more difficult.

MySQL Connector/J

The application uses MySQL Connector/J to connect Java to MySQL through JDBC.

The dependency is declared using:

```
<dependency>
   <groupId>com.mysql</groupId>
   <artifactId>mysql-connector-j</artifactId>
   <version>9.4.0</version>
</dependency>
```

2. Version Control
   The project is maintained using a Github repository.

3. Testing Approach
   Testing is necessary to verify both individual operations and the interaction between application components.

   The following testing categories are relevant.

   Unit Testing

   Unit tests should examine individual classes and methods.

   Examples include:

   - Password hashing and verification.
   - User validation.
   - Laundry-order validation.
   - Complaint creation.
   - Status-transition logic.

   Integration Testing

   Integration tests should examine interactions between application layers.

Examples include:

- User registration and database insertion.
- Login and credential verification.
- Laundry-order creation and garment insertion.
- Complaint submission and database storage.
- Delivery verification and order updates.

Manual Testing

Manual testing should verify the application's user-facing workflows.

For example:

1. Register a customer.
2. Log in as the manager.
3. Approve the customer.
4. Log in as the customer.
5. Submit a laundry order.
6. Update its status.
7. Submit a complaint.
8. Verify the delivery workflow.

4. Testing Results and Evidence

   The implementation was previously reported to have passed Java compilation and basic smoke tests.

   However, a complete live MySQL integration test and full end-to-end workflow test have not been verified.

   The following table distinguishes the known status from tests that remain necessary.

Test	Current Evidence/Status
Java compilation	Previously reported successful
Basic application smoke tests	Previously reported successful
MySQL connection	Not fully verified
User registration with live MySQL	Requires integration testing
Manager account approval	Requires end-to-end verification
Complete laundry-order workflow	Requires end-to-end verification
Complaint and rewash workflow	Requires integration testing
Delivery verification	Requires end-to-end verification
Automated test coverage	No verified coverage report available

5. CI/CD Pipeline

   A complete automated CI/CD pipeline has not been verified as implemented.

A suitable future pipeline would contain:

1. Source-code checkout.
2. Java 17 environment setup.
3. Maven dependency resolution.
4. Compilation.
5. Automated unit tests.
6. Integration tests using a test database.
7. Build artifact generation.
8. Deployment to a designated environment after approval.

Professional, Ethical, and Legal Considerations

1. Data Privacy

   The system handles personal information belonging to students and faculty members.

   Examples include:

   - Names.
   - Institutional identification information.
   - Room numbers.
   - Account credentials.
   - Laundry order histories.
   - Complaint information.

   This information should only be accessible to authorized users.

   The database should not be publicly accessible, and credentials should not be included directly in source-code repositories.

2. Security Measures

   The current implementation includes salted PBKDF2-based password processing.

   This is preferable to storing passwords in plaintext.

   The system also uses user roles to distinguish between students, faculty members, laundry staff, and managers.

   Further security improvements should include:

   - More comprehensive authorization checks.
   - Validation of all user input.
   - Secure handling of database credentials.
   - Protection against SQL injection through parameterized queries.
   - Appropriate database access permissions.
   - Regular backups.

3. Ethical Considerations

   The application should handle user information responsibly.

   Students and faculty members should only be able to access information relevant to their own accounts and orders.

   Laundry personnel should only access information necessary for their operational responsibilities.

   The system should also maintain accurate records of customer complaints and delivery verification.

   Incorrectly recording a delivery or marking an order as completed before the garments are returned could negatively affect customers.

   Therefore, accurate data entry and appropriate verification are important ethical and operational requirements.

4. Legal Considerations

   The system must comply with the relevant institutional policies and applicable data-protection requirements governing personal information.

   MySQL Connector/J is distributed under MySQL's licensing terms. The exact license obligations should be checked against the selected distribution and release.

5. Professional Responsibilities

   The development team is responsible for maintaining accurate documentation, testing the application, identifying known limitations, and communicating incomplete functionality honestly.

   The project should not be presented as a fully operational university service until the required integration, security, and acceptance tests have been completed.

Project Management and Feasibility

1. Development Approach

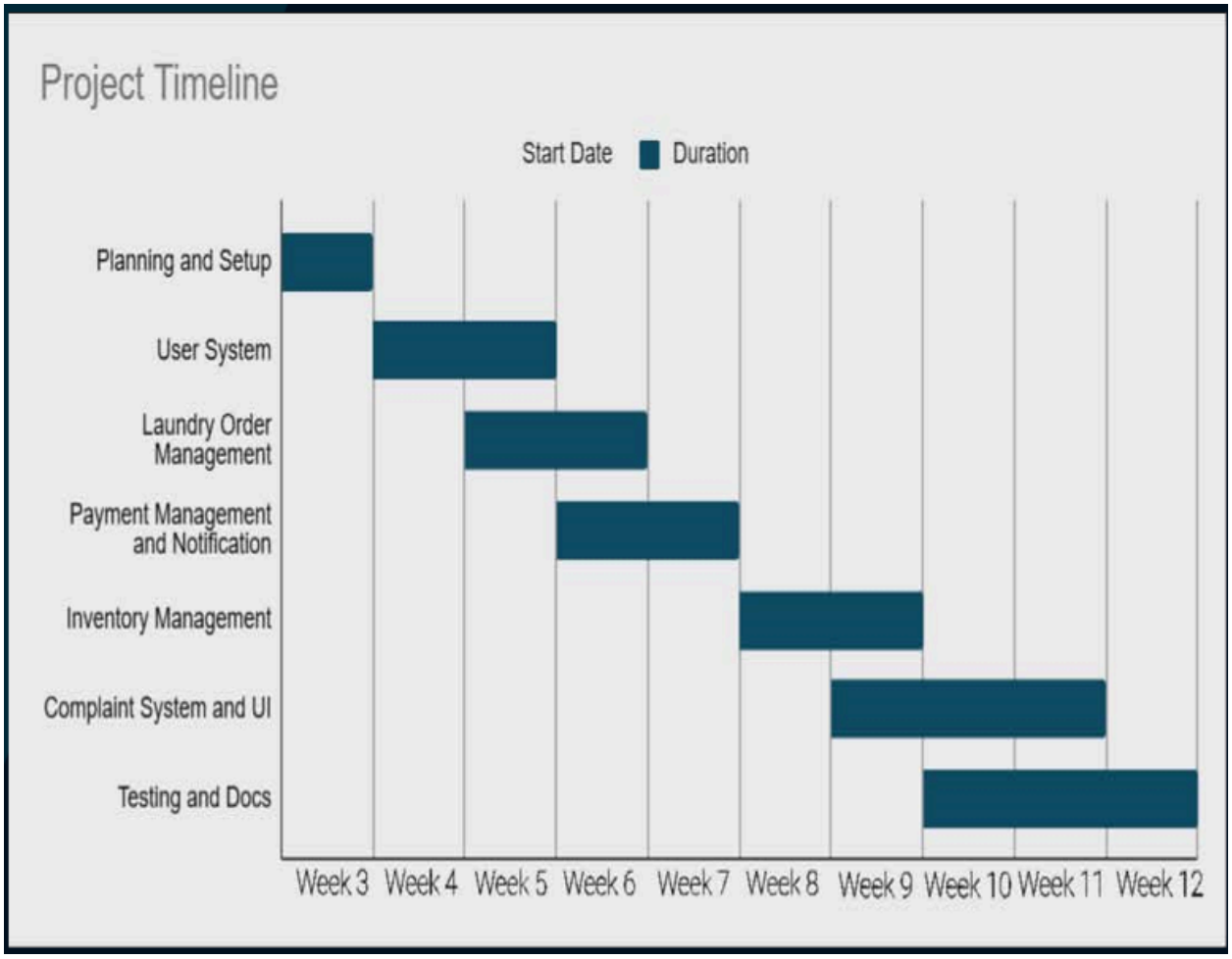
   The project follows a staged software-development approach.

   The major activities include:

   1. Problem identification.
   2. Requirements analysis.
   3. SRS preparation.
   4. UML and architectural design.
   5. Java implementation.
   6. Database integration.
   7. Testing and debugging.
   8. Documentation and presentation preparation.

This approach allows the project to progress from understanding the problem to developing and evaluating a working software solution.

2. Project Timeline



3. Scope and Schedule Changes

During implementation, additional attention was required for Java package organization, Maven dependency configuration, and database connectivity.

These issues affected the integration process and required troubleshooting.

4. Cost Estimation

The project primarily uses established development tools and locally hosted software.

Resource	Purpose	Estimated Cost
Java JDK	Application development	No licensing cost for the selected OpenJDK distribution
IntelliJ IDEA	Development environment	Free Community Edition available
Maven	Dependency management	Free
MySQL Community Edition	Database management	No licensing cost for the Community Edition

MySQL Connector/J	Database connectivity	Subject to the selected distribution's license
Git	Version control	Free
Computer and internet	Development and testing	Existing infrastructure assumed
Hosting/server	Future deployment	Depends on deployment requirements

No paid hosting, payment gateway, or external notification service has been included in the current implementation.

5. Technical Feasibility

   The project is technically feasible because it uses established technologies and does not require specialized hardware.

   Java provides the application logic, MySQL stores persistent data, and JDBC connects the two.

   Maven simplifies dependency management, while IntelliJ IDEA provides the development environment.

   The main implementation requirements can be developed and tested using ordinary development computers.

   However, a production deployment would require additional work involving server configuration, backup procedures, access control, and integration testing.

Teamwork and Individual Contributions

1. Team Composition

Team Member	Student ID
Ibrahim Inyass	230042148
Tahmid Hafiz	230042136

2. Coordination and Integration

   The project contains several interdependent modules.

   The user-management component provides the identity and authentication information required by other modules.

   The laundry-operations component depends on customer information when creating and managing orders.

   The customer-support component depends on user and order information when recording complaints and notifications.

   These dependencies require consistent communication during implementation.

   For example, changes to the `User` class may affect services and repositories in other packages.

Similarly, changes to `LaundryOrder` may affect garment management, delivery processing, and complaint handling.

3. Handling Disagreements and Blockers

   The main technical integration challenges documented during development involved package declarations, Java configuration, Maven dependencies, and MySQL connectivity.

   These issues required checking project structure, classpaths, and database configuration.

4. Fair Participation

   Each team member's contribution should be identifiable through task records and version-control history.

   Maintaining clear responsibilities and communicating integration requirements helps prevent duplicated work and ensures that all members can explain the project.

Results, Limitations, and Future Work

1. Implementation Results

   The project has produced a modular Java application with a MySQL database design.

   The implementation includes user-related, laundry-related, and customer-support-related classes.

   It provides a foundation for managing the main activities of the IUT laundry service.

2. Requirement Implementation Status

   The following table distinguishes implemented functionality from incomplete or unverified requirements.

Requirement	Status	Remarks
User registration	Implemented	Registration functionality is present
Account verification	Implemented	Manager approval functionality is present
User authentication	Implemented	PBKDF2-based password processing
Role management	Implemented	User roles are represented
Laundry requests	Implemented	Order-management functionality is present
Service selection	Partially verified	End-to-end behavior requires testing
Garment management	Implemented	Garment model and database functionality are present
Laundry status tracking	Implemented	Status-management functionality is present
Laundry history	Partially verified	Complete customer workflow requires testing
Complaint management	Implemented	Complaint-related classes and repository are present
Rewash requests	Implemented	Rewash functionality is present
Delivery management	Implemented	Delivery-related functionality is present
Delivery verification	Implemented	Requires full integration verification

Notifications	Partially implemented	Notifications are stored in the database
Payment processing	Not implemented as a complete feature	No verified payment gateway or complete payment workflow
External email/SMS notifications	Not implemented	No external notification integration
Administrative reports	Partially implemented / unverified	Complete reporting functionality has not been established
Web-based interface	Not implemented	Current interface is console-based

3. Known Limitations

   Several limitations remain.

   Console-Based Interface

   The application uses a console interface rather than a graphical or web-based interface. This limits usability for customers who prefer a more accessible visual interface.

   Database Dependency

   The application requires a running MySQL server. It cannot operate as a fully independent application without database connectivity.

   External Notifications

   Notifications are stored in the database, but email and SMS delivery are not implemented.

   Payment Processing

   The current implementation does not provide a complete integrated payment gateway.

   Testing

   Compilation and basic smoke testing have previously been reported, but complete live database integration and end-to-end testing remain necessary.

   Authorization

   Role-based functionality exists, but comprehensive authorization checks should be reviewed and tested across all operations.

   Scalability

   The current architecture is suitable as a project-scale application, but a larger deployment would require additional performance, concurrency, and infrastructure evaluation.

4. Future Work

   Several improvements can be considered.

1. Develop a graphical or web-based user interface.
2. Introduce a REST API to separate the frontend from backend services.
3. Integrate a secure online payment system.
4. Add email or SMS notifications.
5. Improve administrative reports and operational analytics.
6. Introduce automated unit and integration tests.
7. Improve authorization and input-validation mechanisms.
8. Add automated database backup and recovery.
9. Evaluate the application under concurrent usage.
10. Improve deployment and CI/CD automation.

5. Overall Evaluation

   The IUT Laundry Management System demonstrates the application of fundamental software engineering concepts to a real institutional problem.

   It provides a modular foundation for managing customer accounts, laundry orders, garments, complaints, and delivery-related activities.

   Although the implementation is not yet a fully verified production system, it demonstrates the feasibility of replacing several manual laundry-management activities with a centralized digital solution.

Glossary and References

1. Glossary

Term	Meaning
API	Application Programming Interface; a mechanism through which software components communicate
CRUD	Create, Read, Update, and Delete operations
Database	An organized collection of information
DBMS	Database Management System
JDBC	Java Database Connectivity; a Java API for database communication
JDK	Java Development Kit
Maven	A tool for managing Java project builds and dependencies
MySQL	A relational database management system
OOP	Object-Oriented Programming
PBKDF2	Password-Based Key Derivation Function 2
Primary Key	A field that uniquely identifies a database record
Foreign Key	A field that establishes a relationship between database tables
Repository	A component responsible for database access
SRS	Software Requirements Specification
UML	Unified Modeling Language
UI	User Interface
CI/CD	Continuous Integration and Continuous Delivery/Deployment
DTO	Data Transfer Object

2. References

[1] Group 17, *IUT Laundry Management System – SRS Presentation*, SPL-II, 2026. Project requirements, stakeholder identification, functional requirements, and user stories.

[2] Oracle, *Java Platform Documentation*. Available at: https://docs.oracle.com/en/java/

[3] Oracle, *Java Database Connectivity (JDBC) Documentation*. Available at: https://docs.oracle.com/javase/tutorial/jdbc/

[4] MySQL, *MySQL Connector/J Developer Guide*. Available at: https://dev.mysql.com/doc/connector-j/en/

[5] MySQL, *Installing Connector/J Using Maven*. Available at: https://dev.mysql.com/doc/connector-j/en/connector-j-installing-maven.html

[6] OWASP Foundation, *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[7] Apache Software Foundation, *Apache Maven Documentation*. Available at: https://maven.apache.org/guides/

[8] JetBrains, *IntelliJ IDEA Documentation*. Available at: https://www.jetbrains.com/help/idea/

[9] MySQL, *MySQL Reference Manual*. Available at: https://dev.mysql.com/doc/